

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 - ANALYTICAL PROBLEM SOLVING

Course Code:	CSA0305	Course Title:	Data Structures
CO Assessed:	CO2 - Manipulation (BL3, BL4)	Assessment:	Assessment Tool 1 - Analytical Problem Solving
Weightage:	30% of CIA	Total Marks:	100
CO2 Statement:	Implement linear data structures such as stacks, queues, and linked lists, and analyze the efficiency of linear and binary search techniques.		
Student Name:	C. Mokshitha Reddy	Reg. No.:	192571054
Section / Batch:		Date:	20/07/2026

Code of Conduct

I, C. Mokshitha Reddy (192571054) (Name / Reg No)
 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action. Signature of the Student:

Mokshi

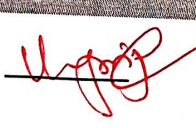
Instructions

- This test contains 5 Analytical problem-solving questions. Total: 100 Marks.
- When a student answer using an Analytical problem-solving, in evaluation the answer should be with following five requirements: Understanding of problem, select appropriate data structure, Design the solution, analyze, Clarity of representation.
- Each student should write 2 questions. No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

Section / Topic	Focus	Q Nos	Marks
Linked Data Structure, Search Data Structure	List Operations, Binary Search	1	50
Stack Data Structure, Queue Data Structures	Stack Notations, Priority Queue	2	50
GRAND TOTAL:		100 Marks	

- Convert the following infix expression into a post fix expression. $B - (C/D + (E \% F * G) / H) * J$. Also, evaluate the given postfix expressions. a) 9 3 8 4 /*+ b) 6 2 + 6 3
Illustrate the queue operation using following function calls of size = 5. Enqueue(21), Enqueue(67), Enqueue(89), Dequeue(), Enqueue(15), Enqueue(30), Enqueue(52), Dequeue(), Dequeue(), Dequeue(), Dequeue().

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20	Demonstrates complete and precise understanding; identifies all constraints and edge cases	Shows clear understanding with minor gaps	Partial understanding; misses some constraints	Misinterprets problem or lacks clarity
Data Structure Selection	20	Selects optimal data structure with strong justification and comparison	Selects appropriate structure with reasonable justification	Selects somewhat suitable structure with weak reasoning	Incorrect or no data structure selection
Algorithm Design	20	Designs efficient, logically sound, and well-structured algorithm	Algorithm is correct with minor inefficiencies	Algorithm is partially correct or lacks clarity	Algorithm is incorrect or missing
Accuracy of Solution	30	Error-free, wellstructured, optimized code/solution	Minor errors but overall functional	Multiple errors; partially working	Non-functional or missing solution
Clarity	10	Exceptionally clear, well-organized, uses diagrams effectively	Clear and organized with minor issues	Somewhat organized but lacks clarity	Poorly organized and unclear
Faculty In-charge		Course Coordinator		Program Director	
Signature: 		Signature: _____		Signature: _____	
Date: _____		Date: _____		Date: _____	

Name : C. Mokshitha Reddy

RegNo : 192571054

Course code: CSA0305

Course Name: Data structures

1. Problem understanding:-

Given:-

1. Convert the infix expression into postfix notation.
2. Evaluate the given postfix expressions using a stack

Infix Expression:-

$B - (C/D + (E * F * G) / H) * J$

Data structure used: Stack

Reason:-

Stack follows the LIFO (last in First out)

Principle, making it suitable for expression Conversion and evaluation.

Selection of Data structure:-

Data structure:-

Stack

Justification:-

- * Stores operators temporarily
- * Maintains operator precedence
- * Handles parentheses correctly
- * Used for postfix evaluation.

Algorithm Design:-

Algorithm for Infix to postfix-

1. Scan expression from left to right
2. If operand $\rightarrow$ add to postfix
3. If '(' push in to stack
4. If ')' $\rightarrow$ pop until '(' is found.
5. If operator:-
 - pop higher / equal precedence operators
 - push current operator
6. After Scanning, pop remaining operators

Conversion:-

Symbol	Stack	Postfix
B		B
-	-	B
(	-(	B
C	-(	BC
/	-(/	BC
D	-(/	BCD
+	-(+ (red arc from D to +)	BCD/
(	-(+(	BCD/
E	-(+(	BCD/E
%	-(+(%	BCD/E
F	-(+(%	BCD/EF
*	-(+(*	BCD/EF%
G (red arc from * to G)	-(+(*	BCD/EF%G
)	-(+	BCD/EF%G*
/	-(+/	BCD/EF%G*

H	$-(+ /$	$BCD EF\% \cdot G^*H$
)	-	$BCD EF\% \cdot G^*H +$
*	- *	$BCD EF\% \cdot G+H +$
J	- +	$BCD EF\% \cdot G^*H +J$
End		$BCD EF\% \cdot GH +J$

Final Postfix Expression:-

$BCD|EF\% \cdot GH|+J-$

Postfix Evaluation:-

a) 9 3 8 4 / * +

Expression

9 3 8 4 / * +

Step	Stack
9	9
3	93
8	938
4	9384

/	932
*	96
+	15

by $62 + 63 *$

Expression:-

6 2 + 6 3 *

Step	stack
6	6
2	62
+	8
3	8 6 3
*	818

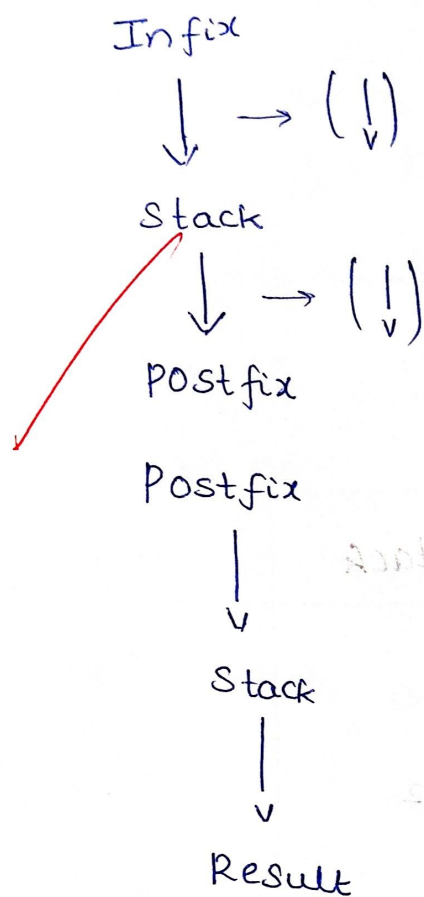
The given postfix expression is invalid - so it cannot be fully evaluated.

Accuracy of solution:-

- Correct evaluation of (a)
- (b) as an invalid postfix expression

Clarity:-

Diagram:-



2. Problem understanding:-

Illustrate queue operations for a queue of size = 5 using;

Enqueue (21)

Enqueue (67)

Enqueue (89)

Dequeue ()

Enqueue (15)

Enqueue (30)

Enqueue (52)

Dequeue()

Dequeue()

Dequeue()

Dequeue()

Data structure used: Queue

Queue principle:

FIFO (First in First out)

Data Structure Selection:-

Data structure: Queue

Reason:

- First inserted element is removed first
- Suitable for scheduling, buffering and process management.

Algorithm Design:-

Enqueue(x)

1. Check if queue is full
2. Insert element at Rear
3. Increment Rear

Dequeue()

1. Check if queue is empty
2. Remove front element

3. Increment Front

Queue Illustration:-

Initial Queue-

Front = -1

Rear = -1

[]

Enqueue (21)

[21]

F

R

Enqueue (67)

[21] [67]

F

R

Enqueue (89)

[21] [67] [89]

F

R

Dequeue ()

Removed = 21

[67] [89]

F

R

Enqueue (15)

[67] [89] [15]

F

R

enqueue(30)

[67] [89] [15] [30]
F R

enqueue(52)

Queue becomes:-

[67] [89] [15] [30] [52]
F R

Dequeue()

Removed = 67

[89] [15] [30] [52]
F R

Dequeue()

Remove = 89

[15] [30] [52]
F R

Dequeue()

Removed = 15

[30] [52]
F R

Dequeue()

Removed = 30

[52]
F R

Final Queue

[52]

Front = Rear

Accuracy of solution:-

Dequeued Elements:

21

67

89

15

30

Remaining Queue:-
52

Clarity:-

Queue Representation:-

Front Rear $\rightarrow$ [21] [67] [89] [15] [30] [52]

$\downarrow$

After all operations

Front $\rightarrow$ [52] $\leftarrow$ Rear

